

ConsensusForge – Project Documentation

Version 1.0 – Concept & Implementation Guide

Date: February 2026

Author: Your name / Chaitanya

License: MIT (recommended for open-source release)-----1. Project Overview

ConsensusForge is an innovative, open-source framework that combines **persistent, autonomous AI agents** with an **evolving multi-LLM consensus mechanism**.

Core Idea

Create personal or task-specific AI agents that run continuously (24/7), maintain long-term memory and state, perform actions via tools/skills, but — unlike single-model agents — make important decisions through a dynamic council of multiple LLMs. The council reviews, debates and synthesizes answers, significantly reducing hallucinations and improving reliability. Over time the system automatically learns which models perform best for which kinds of tasks and evolves the council composition.

Key Goals

- Build trustworthy long-running agents
- Achieve meaningful reduction in hallucinations without fine-tuning or huge RAG
- Make agents measurably smarter the longer they run
- Keep costs very low (using your existing API keys)
- Run locally or on cheap VPS
- Easy to extend and customize

Target Use Cases

- Personal finance/market watcher (NSE/Sensex tracking, alerts)
- Long-term research assistant
- Automated content generator with fact-checking
- Home automation / personal knowledge manager
- Domain-specific bots (education, coding helper, travel planner)

2. High-Level Architecture

[User Interfaces]

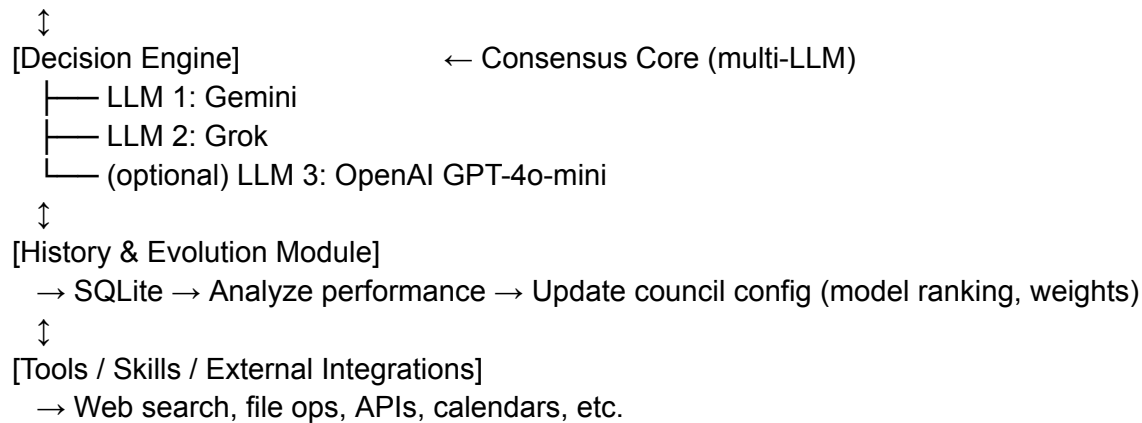
↑ (Telegram / WhatsApp / Web / CLI)

[ConsensusForge Gateway / Agent Runtime]

↑

[Persistent Agent Layer]

← OpenClaw (state, skills, memory, channels)



Components

- **Runtime** → OpenClaw (Node.js) → persistent agents, skills system, 24/7 execution
- **Consensus Core** → Custom logic inspired by karpathy/llm-council → calls multiple LLMs
- **Model Providers** → Google Gemini API + xAI Grok API (+ optional OpenAI)
- **Evolution** → Simple rule-based or statistical re-ranking of models based on history
- **Storage** → Local SQLite for logs, agent memory, consensus history
- **Interfaces** → OpenClaw built-in channels (Telegram strongly recommended)

3. Technology Stack (Zero / Minimal Cost)

Layer	Technology	Cost	Why chosen
Agent runtime	OpenClaw (npm package)	Free ▾	Persistent agents, skills, channels
Language	Node.js (main) + Python (council)	Free ▾	OpenClaw is Node, council easier in Py
LLMs	Gemini API + Grok API (+ optional OpenAI)	Your existing k... ▾	Fast, strong, diverse opinions
Consensus logic	Custom + inspired by llm-council	Free ▾	Multi-LLM review & synthesis
Database	SQLite	Free ▾	Lightweight, local
Communication	Telegram Bot (via OpenClaw)	Free ▾	Easiest persistent interface
Optional extras	Docker (sandbox), PM2 (process mgmt)	Free ▾	Security & reliability

4. Step-by-Step Build GuidePhase 1 – Environment Setup (30–60 min)

1. Install prerequisites

- Node.js 18+ & npm
- Python 3.10+
- Git

Install OpenClaw

npm install -g openclaw

2. openclaw onboard

- Choose **Custom Provider** → you'll add API calls later
- Select **Telegram** channel → create bot via @BotFather → paste token
- Skip LLM provider for now (we override it)

Create project folder

mkdir consensus-forge

3. cd consensus-forge

Clone & prepare LLM Council base (optional inspiration)

git clone https://github.com/karpathy/llm-council.git council-base
cd council-base

4. pip install -r requirements.txt

Phase 2 – Build the Consensus Core (Python – 2–4 hours)

Create file `council.py`

```
import os
import json
from google import genai          # pip install google-generativeai
from xai_sdk import Client as GrokClient  # pip install xai-sdk
from openai import OpenAI         # pip install openai

# Load keys from .env or environment
GEMINI_KEY = os.getenv("GEMINI_API_KEY")
GROK_KEY = os.getenv("GROK_API_KEY")
OPENAI_KEY = os.getenv("OPENAI_API_KEY")

genai.configure(api_key=GEMINI_KEY)
grok_client = GrokClient(api_key=GROK_KEY)
openai_client = OpenAI(api_key=OPENAI_KEY)

DEFAULT_COUNCIL = [
    {"name": "gemini", "model": "gemini-1.5-flash"},
    {"name": "grok", "model": "grok-beta"},
```

```

    # {"name": "gpt", "model": "gpt-4o-mini"} # uncomment when wanted
]

def call_llm(model_info, prompt):
    name = model_info["name"]
    if name == "gemini":
        model = genai.GenerativeModel(model_info["model"])
        resp = model.generate_content(prompt)
        return resp.text
    elif name == "grok":
        chat = grok_client.chat.create(model=model_info["model"])
        chat.append({"role": "user", "content": prompt})
        resp = chat.complete()
        return resp.content
    elif name == "gpt":
        resp = openai_client.chat.completions.create(
            model=model_info["model"],
            messages=[{"role": "user", "content": prompt}]
        )
        return resp.choices[0].message.content
    return "Error: unknown model"

def run_council(query, council=DEFAULT_COUNCIL, use_third=False):
    responses = []
    active_council = council if not use_third else council + [council[2]] if len(council) > 2 else council

    for model in active_council:
        try:
            answer = call_llm(model, query)
            responses.append({"model": model["name"], "answer": answer})
        except Exception as e:
            responses.append({"model": model["name"], "answer": f"Error: {str(e)}"})

    # Simple synthesis (can be improved)
    combined = "\n\n".join([f"[{r['model']}]: {r['answer']}" for r in responses])
    chairman_prompt = f"Review these answers and produce the best final response:\n\n{combined}"

    # Use Gemini as chairman (cheap & fast)
    final = call_llm({"name": "gemini", "model": "gemini-1.5-flash"}, chairman_prompt)

    return {
        "query": query,

```

```

    "responses": responses,
    "final": final
}

```

For CLI testing

```

if __name__ == "__main__":
    result = run_council("What is the current status of Indian stock market?")
    print(json.dumps(result, indent=2))

```

Install dependencies:

pip install google-generativeai xai-sdk openai python-dotenv

Create `.env` file:

GEMINI_API_KEY=your-key-here

GROK_API_KEY=your-key-here

OPENAI_API_KEY=your-key-here # optional

Phase 3 – Integrate into OpenClaw (Node.js – 2–4 hours)

Create custom skill in OpenClaw

Location: `~/.openclaw/skills/consensus.md`

ConsensusForge Skill

Description: Run multi-LLM consensus for important decisions

Trigger: when asked to analyze, decide, summarize uncertain info, or command contains "council" / "verify"

Steps:

1. Extract the decision/query from user message
2. Call Python council script: `python3 /path/to/consensus-forge/council.py --query "{query}"`
3. Parse JSON output

1. 4. Return final consensus + brief explanation of votes

Create bridge script `call_council.js` (in project folder)

```

const { exec } = require('child_process');
const util = require('util');
const execPromise = util.promisify(exec);

```

```

async function runCouncil(query) {
  try {
    const cmd = `python3 council.py --query "${query.replace(/"/g, "\\\"")}"`;
    const { stdout } = await execPromise(cmd, { cwd: '/path/to/consensus-forge' });
    return JSON.parse(stdout);
  } catch (err) {
    return { error: err.message };
  }
}

```

2. `module.exports = { runCouncil };`
3. Modify OpenClaw agent logic (custom SOUL.md or middleware)
Tell agent to use consensus skill for analysis tasks.

Phase 4 – Add Evolution (1–2 hours)

Create `evolve.py` (run periodically via cron / OpenClaw scheduler)

Simple version: count how often each model was closest to final

Later: add user thumbs-up/down feedback

Phase 5 – Testing & Deployment

- Test via Telegram: "Use council to analyze Nifty 50 trend"
- Run on VPS (DigitalOcean ₹400/month) or home PC with PM2 / Docker
- Sandbox strongly (Docker recommended)

5. Next Steps & Roadmap

- Add user feedback loop (thumbs up/down in Telegram)
- More advanced synthesis (LLM ranker + weighted voting)
- Auto-switch council based on task type
- Publish to GitHub
- Add local models (Ollama) as fallback

Good luck building ConsensusForge!

Start small → get Gemini + Grok working → then add evolution → then polish interface.

Let me know which part you want to deep-dive into next (e.g. more detailed Node integration, evolution logic, security).